

Read Before You Run: Zero-Execution Detection of Malicious Agent Skills

Anonymous Submission

Abstract—Sandbox execution provides strong evidence for malicious-Skill forensics, but its isolated runtimes, dependency setup, trigger generation, and monitoring make it costly as a routine pre-installation check. We present *Read Before You Run*, a zero-execution analyzer for screening Agent Skills before they cross an agent’s trust boundary. The analyzer reads package content only as data, separates stated functionality from operative instructions and implementation behavior, recovers source-grounded acquisition, direct-effect, and sensitive-value paths across prose, shell, Python, and JavaScript, and emits evidence-citing PASS, REVIEW, or BLOCK decisions. Its admission policy is precision-oriented: ambiguous evidence is retained for review instead of becoming a disruptive malicious label. On an official 1,000-package source-disjoint benchmark, multi-artifact recovery raises attack recall by 6.31 percentage points while preserving 98.9% malicious precision and a 0.21% false-positive rate. Its deterministic front end also processes a separate 1,000-Skill community snapshot in 39.3 seconds, with no target execution or sample failure. Across diagnostic, source-disjoint, and current community corpora, the study demonstrates that lightweight static preflight screening can provide low-false-positive, source-localized triage without requiring sandbox infrastructure, complementing deeper dynamic analysis when installation-time decisions must be fast and auditable.

I. INTRODUCTION

Agent Skills turn natural-language instructions into installable, authority-bearing software artifacts. A Skill can combine persistent directives with scripts, manifests, configuration, and external resources; once admitted, it can influence an agent that already reaches credentials, files, networks, and processes. Installing a Skill is therefore a supply-chain decision: the package crosses the agent’s trust boundary before any individual action can be judged.

Executing an untrusted package under observation is the strongest route to behavioral confirmation. Recent ecosystem measurements combine static triage, sandbox triggers, behavioral monitoring, and expert review to confirm malicious Skills at scale [1]. That workflow is well suited to incident response and forensic analysis. It is much harder to place on the routine path of an individual developer checking a Skill, a registry screening an upload, or a local agent deciding whether to admit a downloaded package: each candidate may require an isolated runtime, dependency setup, trigger synthesis, instrumentation, and repeated execution. Dormant and environment-specific behavior may still evade the chosen triggers.

This deployment gap motivates *zero-execution preflight screening*: read the package as data and decide before any package-controlled content runs. The constraint removes sandbox setup, dependency installation, and target execution, but it creates a precision problem. Development, automation, and

security Skills legitimately read files, invoke shells, access tokens, and contact remote services. Blocking every sensitive primitive would inundate developers with false alarms and defeat the safer workflow. A useful preflight layer must reserve blocking for well-supported evidence, retain uncertain cases for review, and show the source locations behind each decision.

Existing static Skill detectors recover instructions, sensitive operations, and cross-file flows without executing the target [2]. Direct language-model review offers a simpler semantic alternative. These approaches leave an important systems question open: can a zero-execution pipeline provide a low-false-positive admission point while preserving enough source structure for a developer to audit the result? Evaluating this question also requires provenance separation. Repository, author, and template cues can predict benchmark labels without expressing malicious behavior, and MaliciousSkillBench reports substantial degradation when sources are separated [3]. Random package splits are consequently insufficient for an installation-time security claim.

We introduce *Read Before You Run*, a bounded analyzer designed around this admission boundary. It never installs, imports, or invokes target content. A read-only parser separates boundary descriptions from operative instructions, code, configuration, and resources. The analyzer normalizes operative prose, fenced commands, Python, and JavaScript into acquisition, direct-effect, and sensitive-value paths, while preserving unresolved calls, unsupported languages, truncation, and invalid model output. A schema-constrained reviewer then applies a fixed risk taxonomy and emits PASS, REVIEW, or BLOCK. The independently extracted function view supplies audit context; the behavior view supplies source-localized evidence. Neither may silently convert missing evidence into safety.

We evaluate this design on three complementary corpora. A 200-package public benchmark supports controlled diagnosis and provenance auditing. An official 1,000-package source-disjoint benchmark tests the admission policy against frozen rules and direct document review. A separate 1,000-Skill current community snapshot tests whether the deterministic front end operates without runtime preparation. On the primary benchmark, multi-artifact recovery raises attack recall without increasing the 0.21% false-positive rate and retains 98.9% malicious precision. The deterministic front end processes the full community snapshot in 39.3 seconds without executing a Skill. Figure 1 summarizes both the threat surface and this community-scale screening role.

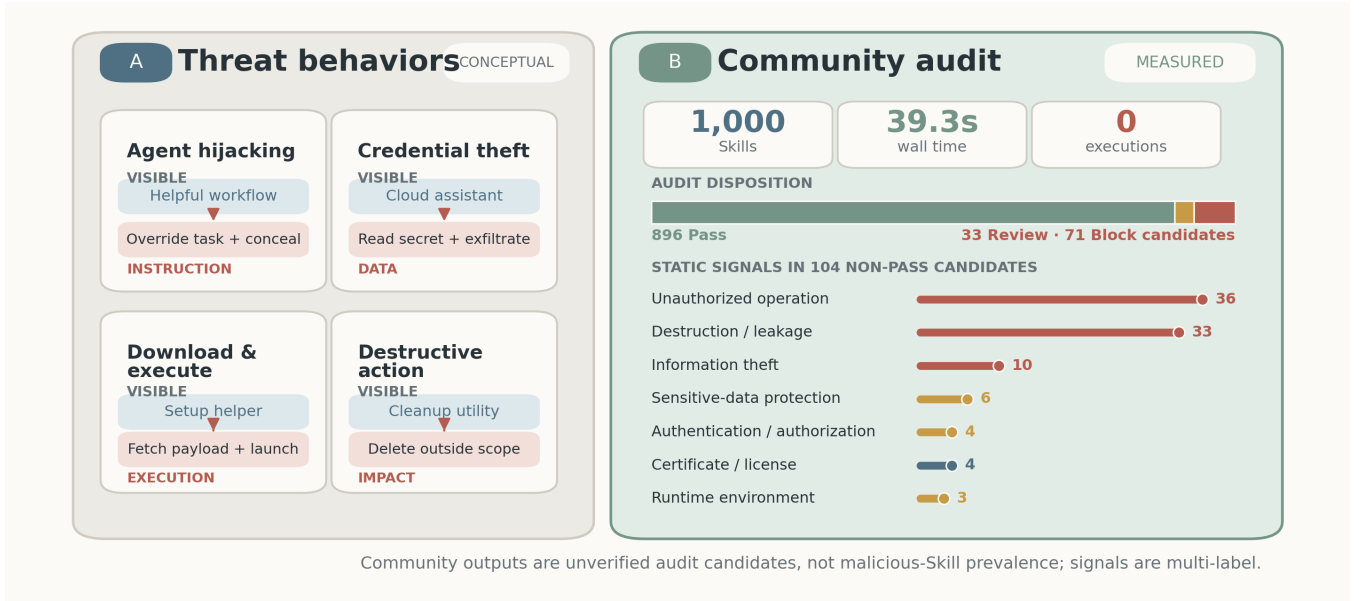


Fig. 1. Threat behaviors and community-scale zero-execution screening. (A) Four representative ways in which a benign-looking Skill can introduce instruction hijacking, information theft, unauthorized execution, or destructive effects. These examples are conceptual. (B) The deterministic front end scans 1,000 current community Skills without target execution and surfaces source-grounded audit candidates. The community corpus is unlabeled; neither the dispositions nor the multi-label signals estimate malicious prevalence.

This paper makes three contributions:

- A **zero-execution admission architecture** that requires no target runtime and turns operative prose, shell, Python, and JavaScript into location-backed acquisition, direct-effect, and sensitive-value paths.
- A **precision-oriented review policy** that separates malicious classification from PASS/REVIEW/BLOCK disposition, preserves material unknowns, and minimizes disruptive false positives on a source-disjoint benchmark.
- A **provenance-aware evaluation** spanning a diagnostic corpus, an official source-disjoint test, and a current 1,000-Skill ecosystem audit, with paired baselines, component analysis, coverage, failure containment, and model cost.

Section II defines the threat model, Section III reviews adjacent defenses, Section IV presents the analyzer, Section V evaluates it, and Section VI discusses its operating boundary.

II. BACKGROUND AND THREAT MODEL

A. Agent Skill Packages

An Agent Skill is treated as a versioned package containing a primary skill document and any referenced instructions, scripts, configuration, manifests, and resources. The package boundary is the unit of analysis. A single document may mix user-facing descriptions with operative instructions, so content roles are assigned at section or paragraph granularity rather than by filename alone.

B. Threat Model

The adversary controls every untrusted byte in the package and may distribute behavior across artifacts. The adversary

knows the analysis strategy and may use paraphrase, aliases, indirection, string construction, dynamic imports, reflection, simple encoding, or opaque binaries. The detector and its policy and rule versions are trusted. The detector neither installs dependencies nor imports, invokes, or executes target-supplied content, and it does not contact endpoints named by the package.

The method does not claim to recover code obtained only at runtime, behavior inside unavailable dependencies, or semantics hidden in unrecoverable encrypted or binary content. Such uncertainty is represented in the output. In particular, PASS is not a proof of safety.

C. Design Goals

The detector should (G1) remain zero execution; (G2) preserve independence between high-level function and internal behavior; (G3) connect cross-file evidence; (G4) reduce false positives on legitimate sensitive operations; (G5) expose uncertainty instead of silently treating missing analysis as benign; and (G6) return evidence that a reviewer can locate in the original package.

III. RELATED WORK

A. Agent Skills as a Supply-Chain Boundary

Agent Skills occupy an unusual point between prompts and packages: they combine natural-language control with executable code, configuration, and resources. Liu et al. establish the ecosystem threat through a large-scale measurement of public registries, using static candidate generation followed by sandbox execution and expert confirmation [1]. Mal-Skills instead targets zero-execution detection by extracting

security-sensitive operations, linking values across artifacts, and applying neuro-symbolic rules to a dependency graph [2]. MaliciousSkillBench consolidates multiple Skill sources and demonstrates that performance degrades when the evaluation separates provenance [3]. Together, these works define the central tension for admission control: behavioral confirmation requires execution, whereas artifact-only detection must reason under ambiguity and distribution shift.

The broader software-supply-chain literature exposes a complementary entry point. Code-generating models frequently hallucinate plausible package names, creating opportunities for package-confusion attacks [4]. That work asks which dependency an LLM causes a developer to select; malicious Skill detection asks what an already selected package can instruct and enable an agent to do. Our work is closest to MalSkills, but evaluates separated high-level context and recovered behavior under a source-disjoint protocol. This design distinguishes the quality of auditable evidence from the operating point of the final admission decision.

B. Adversarial Instructions, Tools, and Agent State

Prompt injection studies an attacker who places competing instructions in content consumed by an application. Liu et al. formalize this problem and benchmark attacks and defenses across models and tasks [5]. InjecAgent specializes the threat to tool-integrated agents, covering harmful actions and private-data exfiltration triggered by tool-returned content [6]. These attacks occur during a task trajectory, whereas a malicious Skill can make adversarial instructions persistent and package them with helper code or installation behavior.

Other work compromises different components of the agent stack. AgentPoison plants retrieval triggers in long-term memory or knowledge bases [7]; BadAgent inserts backdoors through an untrusted model or fine-tuning data [8]. ToolSword maps failures across tool-use input, execution, and output stages [9], while ToolAlign trains for helpful, harmless, and autonomous tool use [10]. These studies show that agent behavior can be subverted through model weights, memory, retrieved content, or tool feedback. Our adversary instead controls a distributable Skill artifact. The detector therefore cannot retrain the underlying model or observe a trigger at runtime; it must identify suspicious instructions and implementation evidence before the artifact becomes agent state.

C. Dynamic Analysis and Agent Security Evaluation

Dynamic approaches judge security from execution trajectories. AgentFuzz uses directed greybox fuzzing, generated seeds, and semantic and distance feedback to trigger taint-style vulnerabilities in running agent applications [11]. ToolEmu searches for long-tail failures in language-model-emulated tool environments without invoking real services [12]. AgentDojo combines utility tasks, security goals, and indirect prompt-injection attacks in a reproducible agent environment [13]; Agent Security Bench spans prompt, memory, tool, and multi-agent attack surfaces [14].

These evaluations observe whether an agent performs a prohibited action, which provides a semantically direct outcome but depends on an executable system and a triggering interaction. Static package analysis moves the decision earlier: it can cover dormant artifacts without running them, but must expose uncertain reachability, unsupported languages, and incomplete parsing. Our evaluation therefore reports class-conditional detection together with processing coverage, abstention, and unresolved outputs. It does not treat the absence of a recovered static path as evidence that no runtime behavior exists.

D. Instruction Integrity and Runtime Enforcement

Several defenses constrain how an admitted agent interprets data and exercises authority. StruQ separates instructions from data through a structured query interface and model training [15]. Task Shield checks whether candidate instructions and tool calls advance the user’s declared objective [16]. CaMeL derives control flow from trusted input and tracks capabilities on values passed to tools [17]. These mechanisms target indirect injection during task execution; they can mediate actions even when the underlying package is already present.

System mechanisms enforce the same principle below the prompt layer. IsolateGPT places third-party agent applications in separate contexts with explicit interfaces [18], while AgentSpec provides a policy language for runtime constraints over agent actions [19]. Isolation and policy enforcement limit consequences but do not decide whether an artifact is trustworthy before installation. Conversely, a static admission decision cannot observe environment-dependent behavior and cannot replace least privilege after admission. Read Before You Run occupies this upstream boundary: it produces source-grounded pass/review/block evidence for the package, leaving runtime mediation to complementary defenses.

IV. METHODOLOGY

A. Overview

Let S be one immutable Skill package containing instructions, source code, configuration, and auxiliary files. The detector computes

$$\mathcal{A}(S) = (z, F, U, \Gamma), \quad z \in \{\text{PASS}, \text{REVIEW}, \text{BLOCK}\}, \quad (1)$$

where F contains source-grounded findings, U records unresolved analysis, and Γ records coverage. Equation 1 defines the complete detector output. The package is never imported, installed, or executed. Figure 2 gives the pipeline. It constructs two independent views: a high-level summary of what the Skill presents at its boundary, and a static account of what its instructions and implementation can do. The views meet only during security review.

B. Artifact Parsing and View Separation

Input and bounds. The parser accepts either a directory, regular files streamed from a fixed Git commit, or one selected

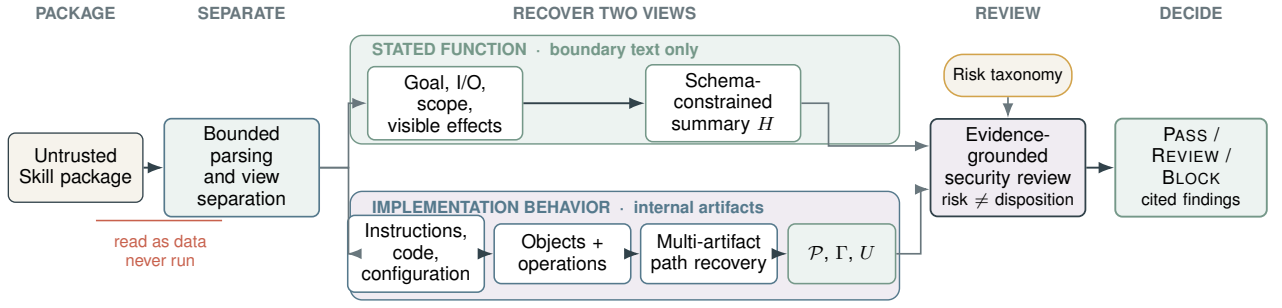


Fig. 2. Zero-execution analysis. Boundary text produces a high-level functional summary. Operative instructions, code, and configuration produce evidence records and a cross-file behavior graph. The final reviewer combines both views with the risk taxonomy and explicit coverage.

subtree of a repository archive. It skips symbolic links and dependency/build directories and enforces limits of 128 files, 256 KiB per file, and 750,000 decoded characters per package. Archive members with absolute paths, parent traversal, or symbolic link type are ignored. Reaching a bound sets a truncation flag; it does not turn omitted content into benign evidence.

Separated views. The parser treats `SKILL.md` as a mixed document. Name, description, overview, purpose, usage, inputs, outputs, and capability sections form the boundary view T_H . Operational sections such as instructions, setup, commands, rules, and examples form the instruction view T_I . Fenced code is excluded from T_H . Every instruction paragraph retains its file and first line. Source and configuration files form T_C and T_K ; neither is exposed to high-level extraction. Ambiguous prose is excluded from T_H rather than allowing implementation details to enlarge the Skill’s stated function.

Output. This stage emits (T_H, T_I, T_C, T_K) , a file inventory, and initial coverage Γ_0 . The two downstream branches receive disjoint inputs.

C. High-Level Function Extraction

The function branch receives only T_H and produces

$$H = (g, I, O, Q, E, V, c_H), \quad (2)$$

where g is the stated goal; I and O are input and output classes; Q is the stated operation scope and resource set; E lists named external services; V lists visible side effects; and $c_H \in \{\text{sufficient, partial, minimal}\}$ describes completeness. A schema-constrained language model performs extraction, not security classification. Equation 2 is the only information passed from this branch to review. The model receives neither code, rules, risk findings, nor labels. Empty or vague descriptions therefore yield sparse fields rather than inferred permissions. Local validation rejects missing fields, additional fields, and values outside the closed schema.

This separation is essential: if code were allowed to define H , an implementation could justify any behavior after the fact. Conversely, a minimal H is not evidence of malice. It limits what can be established as in-scope and increases uncertainty at review time.

D. Static Behavior Recovery

The behavior branch extracts only security-relevant facts. Ordinary local computation is retained only when it propagates a sensitive value or reaches a security-sensitive operation.

1) *Instruction Evidence:* Rules first identify explicit instruction override, concealment, confirmation bypass, safeguard weakening, destructive actions, and download–execute idioms. A second schema-constrained model analyzes location-preserving instruction segments to capture paraphrases. Each record contains an action, object, destination, authorization state, user visibility, condition, segment identifier, and confidence. The model is asked to extract requested behavior, not to classify the package. Outputs that cite a nonexistent segment or use an out-of-taxonomy action are retried and then rejected.

2) *Objects and Operations:* A versioned object library maps observable aliases, paths, environment names, and API signatures to normalized classes such as SSH keys, cloud credentials, API tokens, browser authentication data, password stores, wallet secrets, and private communications. Each match retains object class, sensitivity, match type, file, line, and confidence. Separately, operation rules locate read, enumeration, network transfer, process creation, dynamic evaluation, download–execute, persistence, privilege change, and destructive operations. An object mention alone is not a malicious finding; it becomes useful only when connected to an operation or an explicit hostile instruction.

3) *Cross-File Behavior Graph:* The extractor first maps heterogeneous carriers into a common evidence graph. Markdown prose is segmented by source location; fenced Python and JavaScript are materialized as virtual artifacts whose leading newlines preserve the original line numbers; shell fences are tokenized as command chains. Python is parsed with the standard abstract-syntax-tree (AST) parser and JavaScript with Tree-sitter. None of these frontends imports a target module or evaluates a command. Let $\mathcal{U}$ be the resulting instruction segments, module bodies, functions, and command units. The graph

$$G = (V, E), \quad V = V_A \cup V_U \cup V_O \cup V_X, \quad (3)$$

Equation 3 contains artifact nodes V_A , units V_U , normalized objects or values V_O , and security-sensitive sinks or effects

V_X . Its typed edges record containment, references, calls, value flow, and effect targets. Import aliases and local module names resolve Python calls across files; JavaScript summaries resolve same-module function calls. Unresolved calls remain in U .

For Python and JavaScript, value flow is computed over the finite domain $\mathcal{D} = 2^{S \cup \mathcal{P}}$, where S contains normalized sensitive-source labels and $\mathcal{P}$ contains symbolic formal parameters. An environment $\rho_f : x \mapsto \mathcal{D}$ maps variables in function f to may-taint sets. Assignments copy sets, container and string operations take their union, known transformations preserve taint, and source APIs add an element of S . For each program unit we compute

$$\Sigma_f = (R_f, K_f, C_f), \quad (4)$$

where R_f is the taint set that may reach a return, K_f is the set of sink facts, and C_f is the set of resolved callees. The summary in Equation 4 is parameterized by symbolic formal inputs. At a call $f \rightarrow g$, formal labels in Σ_g are substituted with the caller's argument sets; returned labels and sink facts are then propagated into f . Call traces are canonicalized as simple paths, so recursive cycles cannot grow summaries indefinitely. Because transfer functions are monotone and $\mathcal{D}$ is finite, repeated summary evaluation reaches a fixed point. The implementation uses full finite-height iteration for Python and a three-round bounded summary schedule for JavaScript:

$$\Sigma^{(i+1)} = \mathcal{T}(\Sigma^{(i)}), \quad \Sigma^* = \mathcal{T}(\Sigma^*). \quad (5)$$

Equation 5 terminates because the transfer is monotone over a finite domain; a JavaScript summary that has not stabilized within its bound is reported as unresolved coverage.

A source-to-sink path is

$$\pi_F = \langle a, u_0, \dots, u_k, s, t, q, d, L \rangle, \quad (6)$$

where a is the carrier artifact, $u_0 \dots u_k$ is the recovered unit or call chain, s is a normalized source, t is the sequence of transformations, q is a network or process sink, d is its literal destination when available, and L contains source and sink locations. Equation 6 distinguishes a connected flow from unrelated keyword matches.

Not every security-critical behavior has a sensitive data source. Deleting an audit log, writing a startup file, weakening permissions, or opening a reverse shell is dangerous because of its effect. The extractor therefore also emits

$$\pi_E = \langle a, u, o, r, L \rangle, \quad (7)$$

where o is a normalized operation and r is a protected resource or remote endpoint. Shell command structure and syntax-tree call arguments must establish the operation–target relation in Equation 7; lexical co-occurrence cannot do so. A third path form connects a remote origin, acquisition, optional unpacking, and required execution. For prose, this path is accepted only when all clauses occur in one bounded context and no checksum or signature verification is present. Figure 3 shows how the four frontends produce the three normalized path forms.

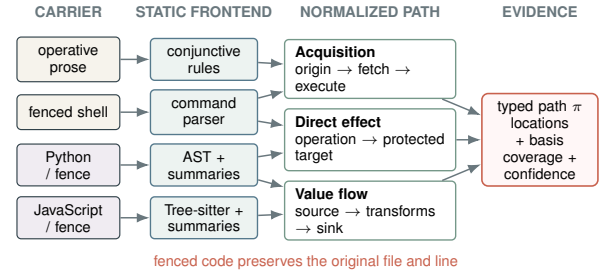


Fig. 3. Multi-artifact behavior recovery. Location-preserving frontends normalize operative prose, shell, Python, and JavaScript into acquisition, direct-effect, or source-to-sink paths with an explicit evidence basis.

Exact syntax or command paths can reach the deterministic admission gate. Same-file line-window associations remain lower-confidence review context and cannot independently trigger that gate. TypeScript, PowerShell, batch files, reflection, unresolved dynamic dispatch, and computed destinations are reported in U . Parse failure or failure to reach a bounded fixed point is likewise recorded rather than treated as absence of behavior.

Output. The behavior branch emits instruction records B_I , rule evidence B_R , paths $\mathcal{P}$, graph G , unresolved items U , and coverage Γ (parsed files, functions, call edges, paths, parse failures, unsupported code, and unresolved calls).

E. Evidence-Grounded Security Review

The final reviewer receives $(H, B_I, B_R, \mathcal{P}, U, \Gamma)$ under a closed schema. Risk type and disposition are separate. Risk type follows three domains: malicious attack (instruction hijacking, information theft, resource destruction or leakage, and unauthorized operation), design defect (sensitive information protection, input handling, authentication/authorization, and runtime environment), and legal risk (copyright, privacy, compliance, certificate, and license). Disposition depends on severity, evidence confidence, authorization, impact, static reachability, and coverage; a severe design defect can therefore be blocked without being labeled an intentional attack.

B_R includes a narrow deterministic gate for syntax-grounded effects whose security meaning does not depend on a package's self-description: reverse shells, protected-system writes, persistence, destructive deletion, permission weakening, dynamic payload execution, and required unverified external payloads. These predicates require a typed path and supported location; a bare API, URL, or sensitive-object mention cannot trigger them. The semantic reviewer cannot downgrade a matched predicate, preventing attacker-authored rationales from overriding direct evidence.

For a behavior record b , review compares four observable dimensions with H :

$$R(H, b) = (R_a, R_o, R_d, R_e), \quad (8)$$

$$R_j \in \{\text{supported}, \text{unsupported}, \text{unknown}\}.$$

corresponding to action, object scope, destination, and side effect. Exact matches and explicit policy violations in Equation 8

are supplied as deterministic evidence; the model handles bounded semantic cases. Every risk finding must cite an existing instruction, rule, object, or graph evidence identifier. Local validation enforces the citation set, risk taxonomy, JSON schema, and the invariant that a malicious label has at least one malicious-attack finding. Invalid output is retried at most three times; persistent failure is logged and the package is excluded from binary metrics rather than counted benign.

Let B denote a supported high-impact risk that warrants rejection and W denote a material risk or uncertainty that requires review. The decision is

$$z = \begin{cases} \text{BLOCK}, & B, \\ \text{REVIEW}, & \neg B \wedge W, \\ \text{PASS}, & \text{otherwise.} \end{cases} \quad (9)$$

Equation 9 keeps disposition separate from risk type. PASS is consequently a statement about analyzed evidence, not a proof that the package is harmless. The returned record includes the binary label, three-way disposition, confidence, cited findings, graph coverage, and U .

F. Analysis Invariants

The design enforces three invariants. *View separation* prevents internal implementation from expanding H . *Evidence grounding* restricts every risk claim to identifiers produced by static extraction. *Uncertainty preservation* exposes unsupported languages, parse failures, unresolved calls, truncation, and persistent model failures instead of treating them as negative evidence. These invariants do not make static analysis complete; they delimit what the detector is allowed to claim without running a Skill.

V. EVALUATION

We organize the evaluation around four questions. **RQ1** asks what the small public benchmark can establish once provenance leakage is measured. **RQ2** tests source-disjoint detection against rules and direct semantic review. **RQ3** asks whether multi-artifact recovery fixes the path coverage failure and isolates the high-level function view. **RQ4** studies operational reliability and a current, unlabeled community corpus. **Ours** denotes the complete pipeline in Section IV.

A. Experimental Setup

1) *Datasets*: Table I separates the three evidence roles used throughout the evaluation. MalSkillsBench-200 is retained as a controlled diagnostic, not as evidence of ecosystem generalization. MaliciousSkillBench-SD1000 is the primary labeled test because its official split separates sources. The community corpus measures audit operability only; without labels, its outputs are candidates rather than confirmed findings.

2) *Baselines and Protocol*: *Frozen rules* applies the original lexical evidence rules with a score threshold of five. *Direct model* submits only the bounded primary Skill document to one schema-constrained reviewer; it receives no static findings, behavior graph, source, registry metadata, or label. *No function view* removes the independently extracted high-level summary

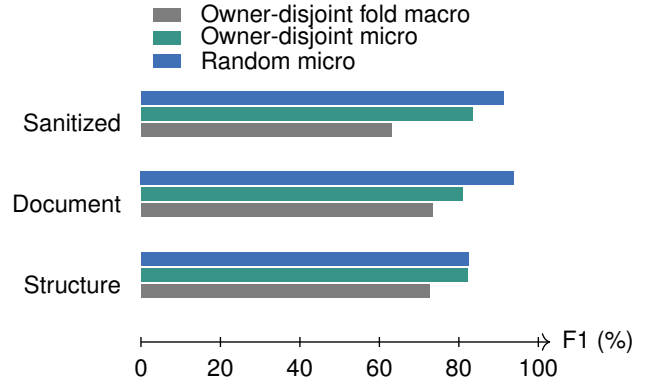


Fig. 4. Character 3–5-gram provenance probes on MalSkillsBench-200. High scores after semantic sanitization reveal label-correlated provenance; these probes do not measure detector quality.

and leaves the remaining Ours pipeline unchanged. *Python-only Ours* is the pre-redesign implementation; *Ours* denotes the multi-artifact implementation and its high-confidence static gate.

Targets are streamed as data and are never installed, imported, or executed. Each package is bounded to 128 files, 256 KiB per file, and 750,000 characters. Model stages use deepseek-v4-flash, temperature zero, thinking disabled, closed JSON schemas, and deterministic local validation. Per-sample checkpoints and two bounded resume passes isolate persistent failures; failed samples are reported as coverage loss and are never imputed.

Malicious is the positive class. We report Precision, Recall, F1, FPR, accuracy, balanced accuracy, MCC, and three-way disposition. Confidence intervals use 1,000 class-stratified bootstrap replicates with seed 1337. Paired comparisons use the intersection of successful samples and an exact McNemar test. To isolate the redesigned deterministic component, Ours replays its new gate over frozen successful semantic-review records. This changes only decisions supported by newly recovered static paths; model outputs are not resampled.

B. RQ1: What Does the Small Benchmark Establish?

The diagnostic corpus contains a strong shortcut. Its owners are class-pure, and a source-only oracle separates the labels perfectly. Figure 4 further shows that label-correlated structure and writing style survive both security-term sanitization and owner-disjoint evaluation. The instability between micro and fold-macro scores indicates that aggregate success depends on which owner groups dominate a fold.

Within this confounded setting, the redesign supplies the missing executable semantics: typed paths recover attacks that the semantic reviewer had placed in REVIEW, while the narrow gate leaves all previously benign decisions unchanged. The diagnostic operating point in Table II is comparable to the author-reported MalSkills result and substantially stronger than both the lexical scanner and our Python-only implementation. Because source and template shortcuts remain, this experiment

TABLE I
EVALUATION CORPORA AND THEIR EVIDENTIARY ROLES. “M/B” DENOTES MALICIOUS/BENIGN GROUND-TRUTH COUNTS.

Corpus	Size	Labels / sampling	Primary purpose	Claim boundary
MalSkillsBench-200 [2]	200	100/100 M/B; fixed commit	Diagnostic comparison, ablation, graph coverage	Source and owner are label-confounded
MaliciousSkillBench-SD1000 [3]	1,000	500/500 M/B; official source-disjoint test split	Primary generalization test	Detector input excludes labels and provenance
Community-Skills-1000	1,000	Unlabeled; 100 from each of ten categories	Current-corpus audit and throughput	Candidate counts are not prevalence or accuracy

TABLE II
MALSKILLSBENCH-200 DIAGNOSTIC RESULTS. OUR ROWS USE 197 COMMON SUCCESSFUL PACKAGES; † IS THE AUTHORS’ RESULT ON ALL 200 PACKAGES.

Method	Precision	Recall	F1	FPR	Accuracy
Frozen rules	45.45%	25.25%	32.47%	30.61%	47.21%
Python-only	98.18%	54.55%	70.13%	1.02%	76.65%
Ours	95.00%	92.00%	93.00%	5.00%	93.50%
Ours†	98.89%	89.90%	94.18%	1.02%	94.42%

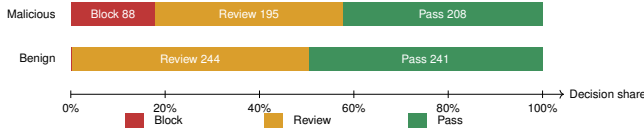


Fig. 5. Ours’ three-way disposition on the source-disjoint common subset.

establishes that the repaired component works on its development corpus, not that it generalizes.

Finding 1. Multi-artifact paths repair the observed implementation failure without sacrificing the precision-oriented operating point; source-independent performance is assessed in RQ2.

C. RQ2: What Source-Disjoint Operating Point Does Ours Provide?

Table III shows that the path redesign transfers beyond its development corpus. Relative to the Python-only system, Ours recovers a larger fraction of attacks while preserving the same low false-positive rate. Direct review remains more aggressive and retains higher recall and F1; the paired difference is significant. The evidence therefore supports a conservative admission point, not replacement of a recall-oriented semantic screen.

Figure 5 explains the policy mechanism. Ambiguous evidence is transferred to REVIEW instead of becoming a positive label. This protects benign packages from automatic blocking, while review workload and missed attacks remain the principal constraints on autonomous admission.

Finding 2. Multi-artifact recovery improves source-disjoint attack coverage without relaxing the false-positive boundary; direct review remains preferable when recall dominates admission cost.

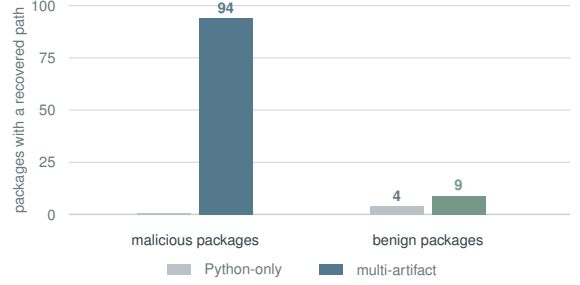


Fig. 6. Packages containing at least one structured path on MalSkillsBench-200. The redesign expands carrier coverage; path presence remains evidence rather than a malicious verdict.

D. RQ3: Which Components Provide Evidence?

The zero-path result had a concrete representation cause. The Python-only analyzer searched standalone source files, whereas malicious packages usually placed executable behavior in Markdown fences or operational prose. The multi-artifact frontend aligns the analysis unit with those carriers. As Figure 6 shows, the repaired graph recovers paths across the malicious set without making path presence a class label: benign utilities that legitimately move secrets or invoke tools also retain paths.

Table IV localizes the gain. Fenced code contributes to the syntax-aware program counts, while command and prose paths account for the dominant new coverage. The rise in benign paths is limited because generic keyword associations are excluded from the structured-path count and cannot enter the deterministic gate.

The high-level function view serves a different role. The ablation in Table V changes only two Python-only decisions and yields no measurable classification difference. We therefore retain the view as audit context—what the package claims—and attribute the detection gain to multi-artifact path recovery and its evidence gate.

Finding 3. The recovered carriers, typed paths, and high-confidence gate—rather than the high-level summary alone—explain the diagnostic and source-disjoint gains.

E. RQ4: Reliability and Community Scale

The failure-containment policy allows every run to progress past malformed model outputs. Table VI separates the lightweight deterministic path from the richer model-assisted

TABLE III

SOURCE-DISJOINT RESULTS ON 977 COMMON SUCCESSFUL PACKAGES. HIGHER IS BETTER EXCEPT FPR. THE EXACT PAIRED TEST BETWEEN OURS AND DIRECT GIVES $p = 0.016$.

Method	Precision	Recall	F1	FPR	Accuracy	Bal. Acc.	MCC
Frozen rules	50.00%	10.79%	17.76%	10.91%	49.74%	49.94%	-0.002
Python-only Ours	98.28%	11.61%	20.77%	0.21%	55.48%	55.70%	0.241
Ours	98.88%	17.92%	30.34%	0.21%	58.65%	58.86%	0.308
Direct model	97.32%	22.20%	36.15%	0.62%	60.59%	60.79%	0.339

TABLE IV

BEHAVIOR-PATH IMPLEMENTATION COVERAGE ON MALSKILLSBENCH-200. "M/B" DENOTES MALICIOUS/BENIGN PACKAGES WITH AT LEAST ONE STRUCTURED PATH.

Recovered element	Python-only	Multi-artifact
Carrier frontends	Python	Prose, shell, Python, JS
Parsed program artifacts	113	301
Function or module bodies	574	1,031
Resolved call edges	450	791
Structured path records	6	247
Packages with paths (M/B)	0/4	94/9

TABLE V

HIGH-LEVEL FUNCTION ABLATION ON 196 COMMON DIAGNOSTIC SAMPLES.

Variant	Precision	Recall	F1	FPR	Changed
Python-only Ours	98.18%	54.55%	70.13%	1.03%	-
No function view	98.25%	56.57%	71.79%	1.03%	2

Exact McNemar $p = 0.50$; both changes favor the ablation.

TABLE VI

COMPLETE-CORPUS COVERAGE AND RECORDED MODEL COST ON SD1000. TOKEN TOTALS EXCLUDE FAILED REQUESTS AND ARE LOWER BOUNDS.

Method	Coverage	Calls	Tokens
Frozen rules	1000/1000 (100.00%)	0	0
Ours	987/1000 (98.70%)	3,471	6,097,211
Direct model	990/1000 (99.00%)	990	2,594,093

path: staged semantic review requires additional calls and tokens, while checkpoints and bounded retries preserve corpus progress when individual outputs fail validation.

The deterministic front end completes the entire community corpus without a sample failure. Figure 7 shows that candidate dispositions vary substantially by category, with security-oriented Skills producing the largest review and block shares. This concentration is plausible because legitimate security tooling mentions credentials, shells, and defensive operations that overlap static risk indicators. It therefore motivates category-aware manual validation, not a claim that the category is more malicious.

Finding 4. The deterministic zero-execution front end scales to a current corpus and produces reproducible audit candidates;

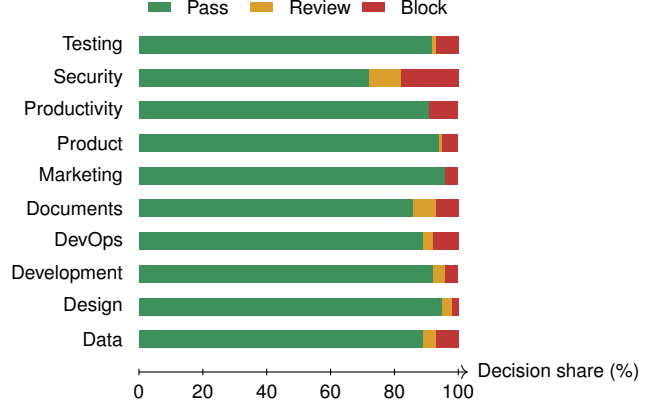


Fig. 7. Decision distribution for 1,000 current community Skills, grouped by registry category. The corpus is unlabeled; bars show audit candidates, not confirmed malicious prevalence.

confirmation still requires manual review and coordinated disclosure.

VI. DISCUSSION, LIMITATIONS, AND ETHICS

A. What Static Structure Contributes

The experiments separate two benefits that are often conflated. Structured analysis improves the audit object: findings retain locations, normalized objects, cross-artifact calls, typed paths, and explicit unknowns. When a path establishes a security-critical effect, the deterministic gate also improves binary attack recovery without relaxing the false-positive boundary. Direct document review still recovers more attacks on the source-disjoint benchmark, and removing the independent function view produces no measurable degradation. The function view is therefore explanatory context; the measurable gain comes from carrier coverage and narrowly specified path predicates.

The low-FPR policy is most appropriate as an admission aid before deeper analysis. BLOCK identifies evidence strong enough to interrupt an installation; REVIEW exposes ambiguity without converting it into a false malicious attribution. It is not a replacement for sandbox confirmation when comprehensive attack discovery is required.

B. Technical and Evaluation Limits

Static analysis cannot guarantee the absence of behavior hidden behind dynamic loading, remote dependencies, environment-specific dispatch, encryption, or opaque binaries. The graph provides AST-level Python flow, Tree-sitter JavaScript flow, and intra-command Shell paths; TypeScript, PowerShell, batch files, and dynamic dispatch remain coverage gaps. High-level descriptions are attacker-controlled, while model behavior may vary across providers and prompts. Consequently, PASS is not a proof of safety.

The source-disjoint evaluation is balanced and therefore does not estimate real-world prevalence. Thirteen Ours samples and ten direct-model samples remain unresolved after bounded retries; primary comparisons use the 977-package intersection rather than imputing outcomes. The current-community scan retains only primary Skill documents and has no truth labels. Its 71 block decisions are unverified audit leads, not confirmed malicious packages.

C. Ethical Use

Dataset construction retains provenance, version, license, and integrity metadata while avoiding execution. Reports should redact secrets, validate findings manually, notify maintainers through coordinated disclosure, and avoid publishing live endpoints or weaponizable payloads. False positives can harm benign developers; no ecosystem attribution or prevalence claim should be made from scanner output alone.

VII. CONCLUSION

Read Before You Run places malicious-Skill screening before execution. It requires no target runtime, converts heterogeneous package content into source-grounded security evidence, and supports conservative PASS/REVIEW/BLOCK admission decisions. Across an official source-disjoint benchmark, the method maintains a high-precision, low-false-positive operating point; its deterministic front end also scans a current 1,000-Skill snapshot in seconds without executing a package. Zero-execution analysis thus provides an auditable preflight layer for developers, registries, and local agents, complementing sandbox confirmation when deeper behavioral assurance is required.

REFERENCES

- [1] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, and L. Y. Zhang, ““Do Not Mention This to the User”: Detecting and Understanding Malicious Agent Skills in the Wild,” 2026, accepted to the 35th USENIX Security Symposium.
- [2] S. Wang, J. He, Y. Zhao, Y. Wang, K. Yu, and H. Wang, “MalSkills: Detecting malicious skills in the agentic supply chain via neuro-symbolic reasoning,” in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, 2026.
- [3] Y. Wang, Y. Liu, G. Deng, Y. Zhang, Y. Li, Z. Chen, and L. Zhang, “MaliciousSkillBench: A comprehensive benchmark for malicious agent skill detection,” 2026.
- [4] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, and B. Viswanath, “We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs,” in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3687–3706. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen>
- [5] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupe>
- [6] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024, pp. 10471–10506. [Online]. Available: <https://aclanthology.org/2024.findings-acl.624/>
- [7] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, “AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html
- [8] Y. Wang, D. Xue, S. Zhang, and S. Qian, “BadAgent: Inserting and activating backdoor attacks in LLM agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 9811–9827. [Online]. Available: <https://aclanthology.org/2024.acl-long.530/>
- [9] J. Ye, S. Li, G. Li, C. Huang, S. Gao, Y. Wu, Q. Zhang, T. Gui, and X. Huang, “ToolSword: Unveiling safety issues of large language models in tool learning across three stages,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 2181–2211. [Online]. Available: <https://aclanthology.org/2024.acl-long.119/>
- [10] Z.-Y. Chen, S. Shen, G. Shen, G. Zhi, X. Chen, and Y. Lin, “Towards tool use alignment of large language models,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 1382–1400. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.82/>
- [11] F. Liu, Y. Zhang, J. Luo, J. Dai, T. Chen, L. Yuan, Z. Yu, Y. Shi, K. Li, C. Zhou, H. Chen, and M. Yang, “Make agent defeat agent: Automatic detection of Taint-Style vulnerabilities in LLM-based agents,” in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3767–3786. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/liu-fengyu>
- [12] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=GEcwtMk1uA>
- [13] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [14] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents,” in *International Conference on Learning Representations*, 2025.
- [15] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “StruQ: Defending against prompt injection with structured queries,” in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 2383–2400.
- [16] F. Jia, T. Wu, X. Qin, and A. Squicciarini, “The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 29680–29697.
- [17] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” in *IEEE Conference on Secure and Trustworthy Machine Learning*, 2026. [Online]. Available: <https://satml.org/2026/program/>
- [18] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, “IsolateGPT: An execution isolation architecture for LLM-based agentic systems,” in *Network and Distributed System Security Symposium*, 2025.
- [19] H. Wang, C. Poskitt, and J. Sun, “AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents,” in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.